



B ve B+ Ağaçlarının Modern Sistemlerdeki Rolü

Ad Soyad : Yavuz Eymen Tan

Öğrenci No : 02240224144

Ders : Veri Organizasyonu

Kurum : İnönü Üniversitesi

Tarih : 19.05.2026

B-TREE • B+ TREE • INDEX

Sunum Akışı

Ödev konusunu örnek sunumdaki gibi kısa başlıklar ve görsel akışla anlatan yapı.

01

Tanım

B ve B+ ağaçlarının temel farkı

02

Çalışma Mantığı

Arama, dengeleme ve yaprak zinciri

03

Kullanım Alanları

Veri tabanları, dosya sistemleri, arama motorları

04

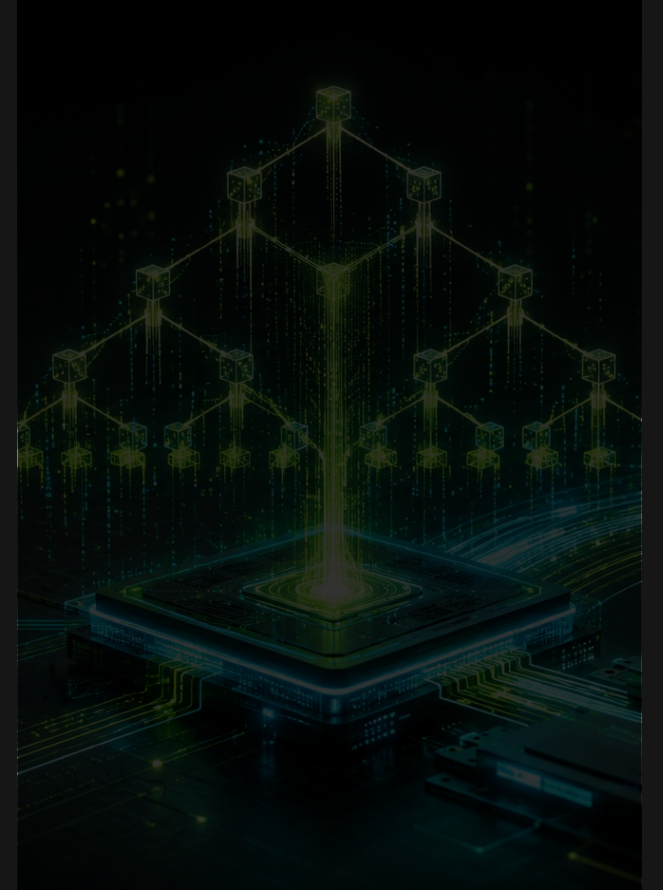
Günlük Benzetme

Kütüphane ve hastane arşivi örneği

05

Yorum

Büyük veri çaığında önem ve gelecek

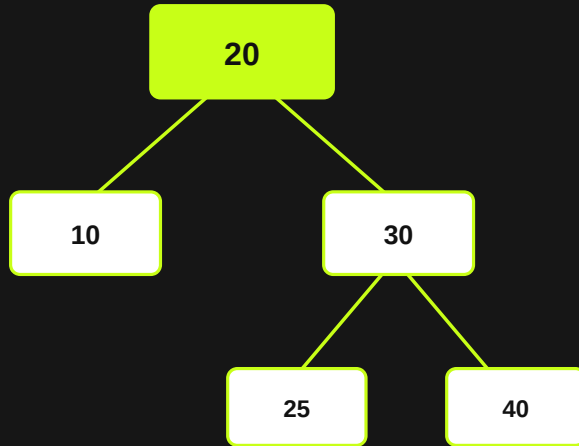


B ve B+ Ağaçları Nedir?

İki yapı da çok yollu, dengeli ve sıralı arama ağacı mantığına dayanır.

B-Ağacı

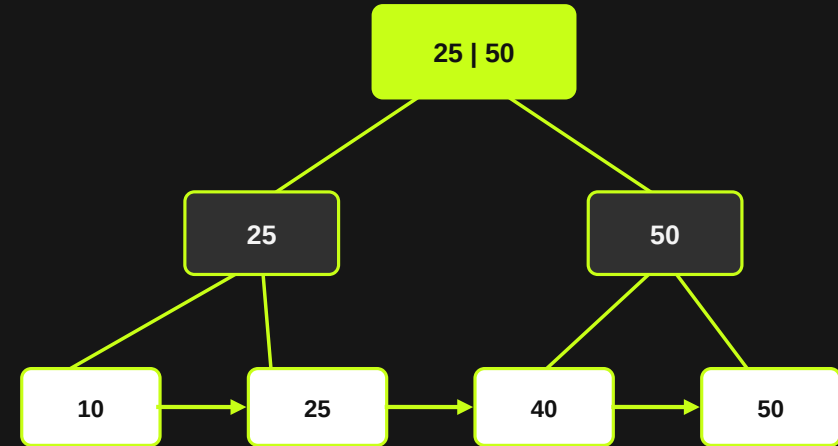
Kendi kendini dengeleyen çok yollu arama ağacıdır. Bir düğümde birden fazla anahtar bulunabilir. Veriler hem iç düğümlerde hem de yapraklarda yer alabilir.



B-Ağacı: veri üst seviyede de bulunabilir

B+ Ağacı

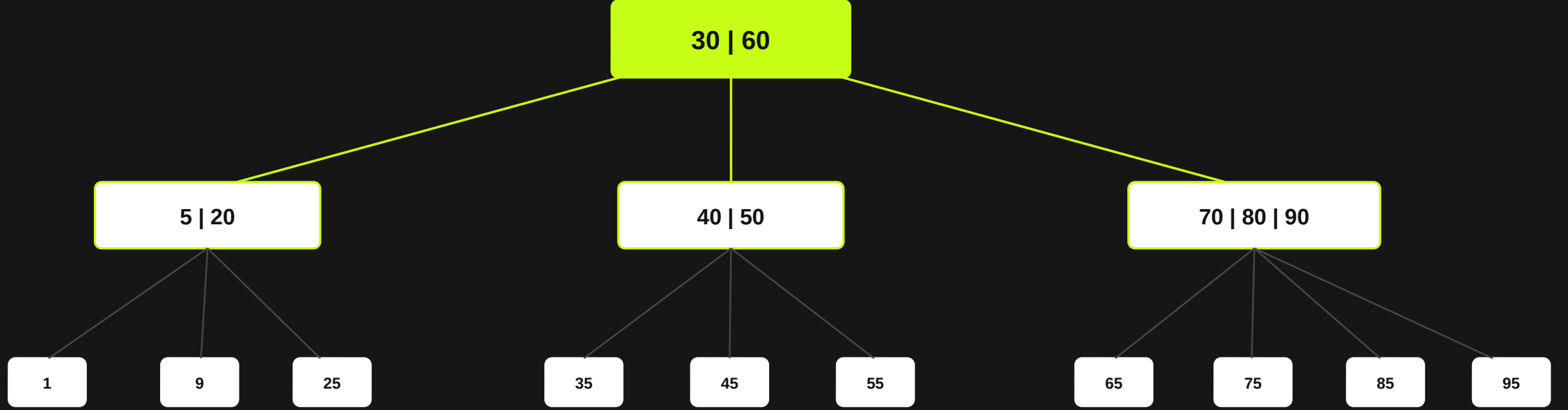
B ağacının geliştirilmiş biçimidir. İç düğümler yalnızca yönlendirme/indeks anahtarlarını taşır. Gerçek veriler yaprak düğümlerde ve yapraklar birbirine bağlıdır.



B+ Ağacı: veri yapraklarda ve yapraklar zincirli

B-Ağacının Çalışma Mantığı

Geniş düğümler sayesinde ağaç kısa kalır; bütün yapraklar aynı seviyede tutulur.



Kısa yükseklik

- Daha az seviye
- Daha az disk erişimi

Sürekli dengeli

- Tüm yapraklar aynı seviyede
- Ekleme sonrası denge yapma

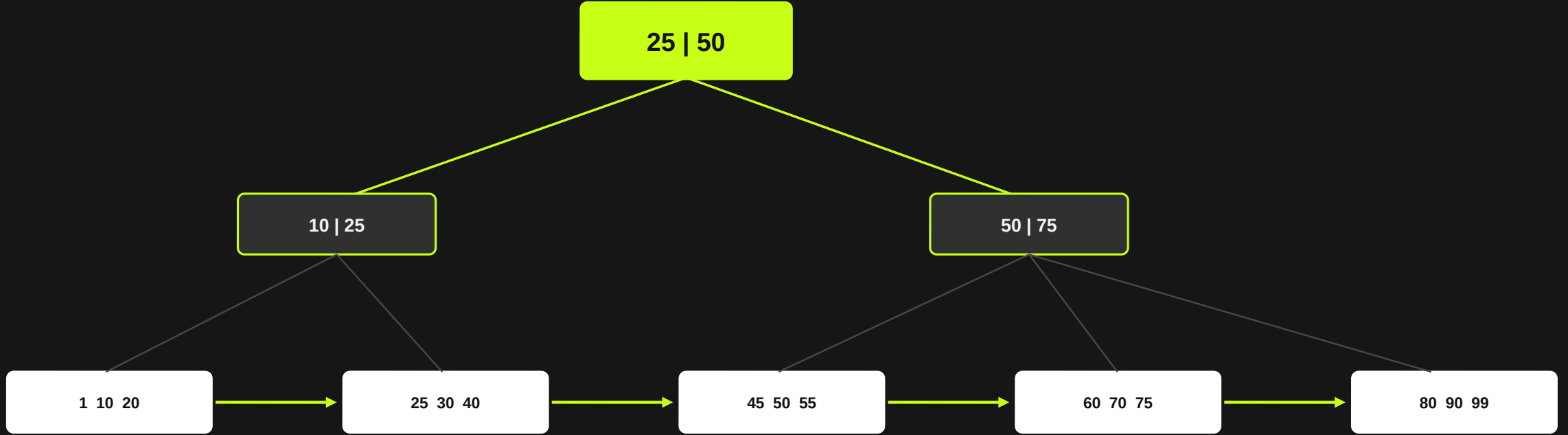
Hızlı arama

- Arama, ekleme ve silme
- Genel karmaşıklık O(log n)

Ders notu: B ağaçları dengeli, çok yollu ve disk erişimini azaltan yapılardır.

B+ Ağacının Çalışma Mantığı

B+ ağacında iç düğümler yönlendirir; gerçek kayıtlar yaprak seviyesinde tutulur.



1. İç düğümler

• Sadece indeks/yönlendirme tutar

2. Yapraklar

• Gerçek kayıt veya kayıt adresi burada bulunur

3. Bağlantı

• Yaprak zinciri aralık sorgusunu hızlandırır

Ders notu: B+ ağacında kayıtlar yalnızca yaprak düğümlerde saklanır ve yapraklar bağlantılı liste gibi birbirine bağlanır.

B-Ağacı ve B+ Ağacı Karşılaştırması

Aynı aileden gelseler de veri saklama ve aralık tarama davranışları farklıdır.

Kriter	B-Ağacı	B+ Ağacı
Verinin konumu	İç düğümde veya yaprakta olabilir.	Gerçek veri yaprak düğümlerdedir.
Arama sonucu	Bazı aramalar üst seviyede bitebilir.	Arama çoğunlukla yaprağa kadar iner.
Aralık sorgusu	Sıralı tarama daha maliyetli olabilir.	Yaprak zinciri sayesinde hızlıdır.
Disk kullanımı	Genel amaçlı indekslemeye uygundur.	Veri tabanlarında çok yaygındır.
Benzetme	Her katta tabela ve dosya bulunur.	Üst katlar tabela, alt kat gerçek dosyadır.

Modern Sistemlerde Neden Tercih Edilir?

Modern sistemlerde amaç, çok büyük veriye hızlı ve güvenilir şekilde erişmektir.



Disk I/O Optimizasyonu

- Tek disk okumasıyla çok sayıda indeks anahtarı işlenebilir.

Logaritmik Zaman

- Kayıt sayısı büyüdüğünde arama maliyeti kontrollü artar.

Önbellek ve Denge

- Üst düğümler bellekte tutulur, dengeli yapı yolu kısaltır.

CPU hızlıdır; disk erişimi pahalıdır. B+ Tree bu farkı azaltır.

Veri Tabanlarında Kullanımı

İndeks, tablonun tamamını taramadan hedef kayda ulaşmayı sağlayan hızlı erişim yoludur.

MySQL / InnoDB

- İndeksler B-tree yapısındadır.
- Birincil anahtar clustered index mantığıyla veriyi düzenler.
- Amaç daha az disk erişimiyle satıra ulaşmaktır.

PostgreSQL

- Varsayılan ve en yaygın indeks türü B-tree'dir.
- Eşitlik ve aralık sorgularında kullanılır.
- Sıralama işlemlerini destekler.

MongoDB

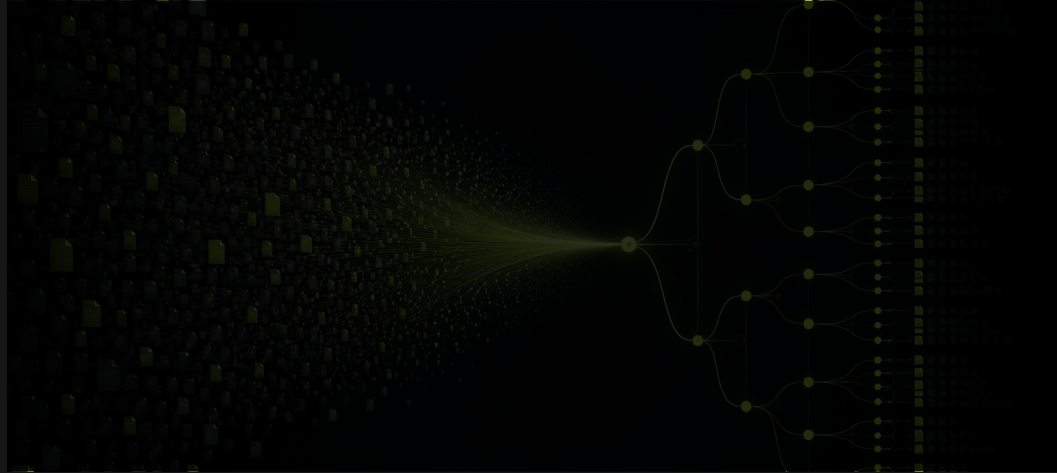
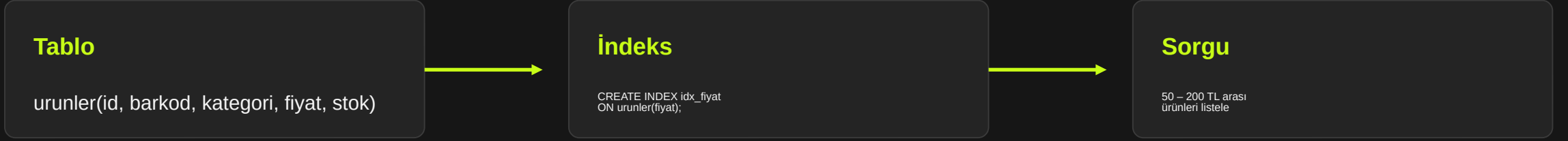
- Belge tabanlı olsa da indeksler sıralı anahtar mantığıyla çalışır.
- B-tree indeks yapısı hızlı sorgulama sağlar.
- Eşitlik ve aralık sorgularında faydalıdır.

Örnek: `SELECT * FROM urunler WHERE fiyat BETWEEN 50 AND 200 ORDER BY fiyat;`

Fiyat alanında B+ Tree indeks varsa veri tabanı ilk uygun yaprağa iner, sonra yaprak zinciri boyunca okur.

Somut Veri Tabanı Örneği: Ürün Arama

Bir stok veritabanında barkod, kategori, fiyat ve stok alanları üzerinden hızlı arama yapılır.



Bu indeksin faydası nedir?

- Tüm ürünleri tek tek okumaz.
- Fiyatlar sıralı tutulduğu için ilk uygun noktaya hızlı iner.
- Aralık içindeki kayıtları sırayla okur.
- Rapor ve listeleme ekranları hızlanır.

Dosya Sistemlerinde Kullanımı

Dosya sistemleri de büyük bir “adres bulma” problemidir: dosya adı, klasör ve blok adresi hızlı bulunmalıdır.



NTFS / Windows

- MFT ve dizin yapıları dosya kayıtlarını hızlı bulmak için indeks mantığından yararlanır.

EXT4 / Linux

- Büyük dizinlerde HTree adı verilen yüksek dallanmalı dizin indeksleme yapısı kullanılır.

APFS / Apple

- Dosya sistemi yapılarında B-tree düğümleri ve ağaç tabanlı organizasyon kullanılır.

Ortak hedef: milyonlarca dosya içinde hızlı ve güvenilir erişim

Arama Motorları ve Büyük Veri Sistemleri

Büyük sistemlerde veriye yalnızca ulaşmak değil, onu sıralı ve aralıklı biçimde okuyabilmek önemlidir.

Ters İndeks Mantığı

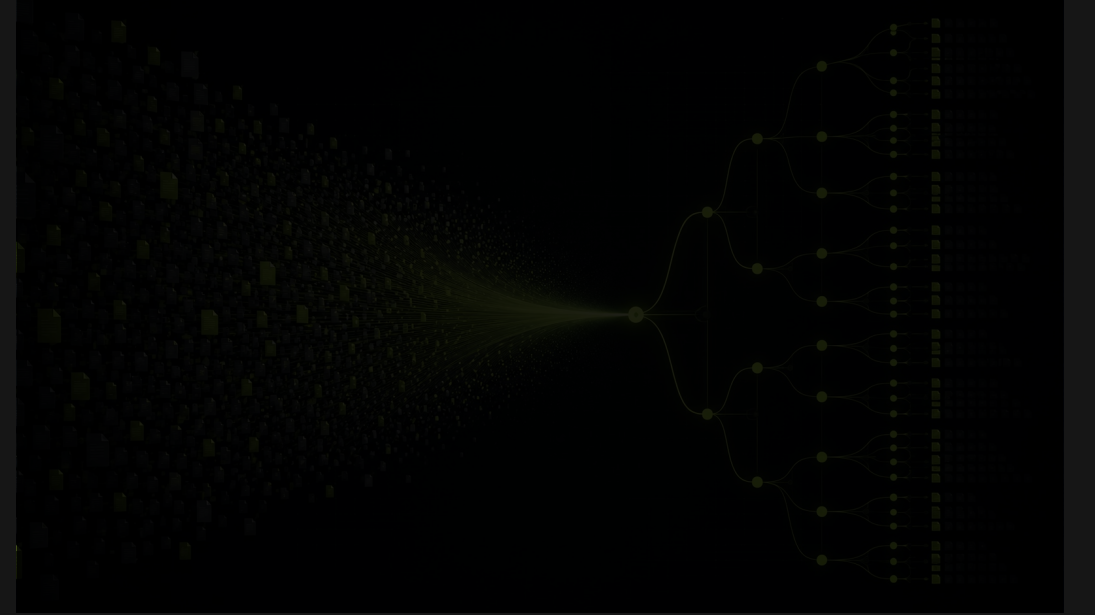
- Kelime → doküman listesi ilişkisi indekslenir.
- Terim sözlüğüne hızlı erişim için ağaç/indeks mantığı kullanılır.

Büyük Veri Analitiği

- Zaman, fiyat veya kullanıcı aralığına göre filtreleme yapılır.
- Sıralı veriye erişim raporlama hızını artırır.

Neden Hash Yetmez?

- Hash tablo tek kayıt bulmada hızlıdır.
- Sıralı gezinme ve aralık sorgularında B+ Tree daha uygundur.



B+ Tree = ilk noktayı bul + yaprak zincirinden sırayla devam et

Günlük Hayat Benzetmesi: Kütüphane Sistemi

Bir kütüphanede kitap bulma süreci, B+ ağacının çalışma mantığını somutlaştırır.



Üst Kat: Yönlendirme

- “A-F arası 1. reyon” gibi tabelalar bulunur.

Alt Kat: Gerçek Kitaplar

- Gerçek kitaplar alfabetik ve sıralı raflarda durur.

Yan Yana Raflar

- Raflar zincir gibi birbirine bağlanmış kabul edilir.

Somut Fayda

“K ile M arasındaki tüm yazarları getir” denildiğinde tüm kütüphane dolaşılmaz. Önce K harfine gidilir, sonra sıralı raflar boyunca M harfine kadar ilerlenir. Bu, B+ Tree’deki yaprak zincirine benzer.

Aralık Sorgularında B+ Tree Avantajı

Tek kayıt aramak ile belirli aralıktaki tüm kayıtları almak farklı problemlerdir.

Nokta Sorgusu

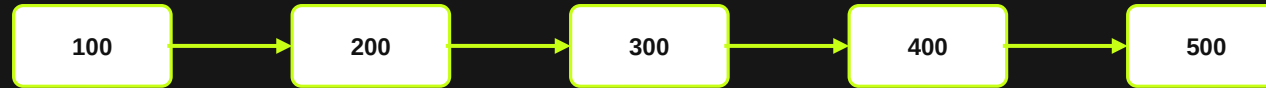
“Barkodu 869123 olan ürünü getir.”
Tek hedef bulunur. B ağacı ve B+ ağacı bu senaryoda hızlıdır.

Aralık Sorgusu

“100 TL ile 500 TL arasındaki ürünleri getir.”
B+ Tree başlangıç değerine iner ve yaprak zincirinde ilerler.

Kökten yaprağa

Yatay tarama



$O(\log n + k)$: hedefe in + k adet sonucu sırayla oku

Büyük Veri Çağında Neden İhtiyaç Duyulur?

Veri büyüdükçe tüm kayıtları sırayla kontrol etmek gerçekçi değildir.

Hash Tablosu: Sınırlı Hız

- Tek kayıt aramada çok hızlıdır.
- Veri hash fonksiyonuyla dağılır.
- Sıralı gezinme doğal değildir.
- Aralık sorgularında veriler dağınık kalabilir.

B+ Tree: Aralık Dostu

- Veri sıralı düzende tutulur.
- Aralık sorguları doğal olarak desteklenir.
- Yaprak bağlantısı kesintisiz tarama sağlar.
- Disk erişimi ve önbellek verimli kullanılır.

Büyük veri = yalnızca bulmak değil, sıralı ve aralıklı okumak

Neden Önemlidir ve Gelecekte Kullanılır mı?

Kendi yorumum: B/B+ ağaçları modern indeksleme düşüncesinin temelidir.

Neden Önemli?

Veri büyüdüğünde hız sorunu ortaya çıkar. B ve B+ ağaçları veriyi sıralı, dengeli ve disk dostu biçimde tutarak arama maliyetini düşürür.

Büyük Veri Çağında

Milyonlarca kayıt içinde filtreleme, sıralama ve raporlama yapılır. B+ Tree bu işlemleri tüm veriyi taramadan yapmaya yardım eder.

Gelecek Yorumu

Kullanılmaya devam edecektir. Donanım hızlansa bile veri miktarı artıyor. Yeni indeks yapıları bile hiyerarşik ve dengeli erişim fikrinden etkilenir.

Sonuç: B+ Tree, veriyi yalnızca saklayan değil, veriye ulaşma şeklini hızlandıran bir organizasyon modelidir.

Genel Sonuç

Ödev sorularının tamamını kapsayan kısa kapanış.

1

Veri tabanlarında

B/B+ ağaçları indeks yapısı olarak kullanılır; tabloyu baştan sona taramayı azaltır.

2

Dosya sistemlerinde

Klasör, dosya adı ve blok adresleri gibi metadataların hızlı bulunmasına yardım eder.

3

Büyük veri sistemlerinde

Sıralı veri, aralık sorguları ve hızlı raporlama için temel bir yaklaşımdır.

4

Günlük benzetme

Kütüphane veya hastane arşivi, B+ Tree'deki yönlendirme ve yaprak zincirini açıklar.

5

Kişisel yorum

Gelecekte donanım değişse bile dengeli indeksleme mantığı önemini koruyacaktır.

Teşekkürler

Kaynakça

Sunum hazırlanırken kullanılan temel kaynaklar.

1. Ders Notu: Hafta 6-7 B Ağaçları, Veri Organizasyonu ders materyali.
2. MySQL 8.4 Reference Manual: InnoDB Index Types / Clustered and Secondary Indexes.
3. PostgreSQL Documentation: Index Types — B-Tree Indexes.
4. MongoDB Manual: Indexes — B-tree based index structure.
5. Apple File System Reference: B-tree node constants and file system structures.
6. Linux EXT4 / HTree teknik açıklamaları: yüksek dallanmalı dizin indeksleme.
7. Bayer & McCreight, “Organization and Maintenance of Large Ordered Indices”, 1970.